Comprehensive Study Guide

NARX-Based False Data Injection Attack Detection

in EV Charging Networks

Two-Stage Isolation Forest + CUSUM Pipeline

Sudarshan Sudhakar
IIITDM Kancheepuram — Information Security, Semester 6

April 7, 2026

Contents

1 The Problem: What Is Being Solved? 3
1.1 Threat Context . . . 3
1.2 Why Existing Approaches Fail . . . 3
1.3 The Solution Paradigm: Estimate Then Detect . . . 3

2 The Dataset: Caltech ACN-Data 3
2.1 Source and Content . . . 3
2.2 Preprocessing: From Sessions to Time-Series . . . 4

3 The NARX Neural Network — Deep Dive 4
3.1 What NARX Stands For . . . 4
3.2 Architecture Specification . . . 5
3.3 The 15 Exogenous Features . . . 5
3.4 Open-Loop vs. Closed-Loop: The Critical Distinction . . . 5

4 Core Concepts: Non-Linearity and Multi-Modality 6
4.1 Non-Linear Data in EV Charging . . . 6
4.2 Multi-Modal Data in FDI Detection . . . 6

5 Technical Specifics: Weights and Gradients 6
5.1 Xavier (Glorot) Uniform Initialization . . . 6
5.2 NARX Input Window Construction (`dataset.py`) . . . 7
5.3 Why NARX for EV Charging? . . . 7

6 Training Pipeline (`src/train/train.py`) 7
6.1 Training Hyperparameters . . . 7
6.2 Training Flow . . . 7
6.3 Target Benchmark . . . 8

7 Evolution of the Detection Layer 8
7.1 Iteration 1: Global IQR — The Paper Baseline (`evaluate.py`) . . . 8
7.2 Iteration 2: Session-Aware IQR (`evaluate.py`) . . . 9
7.3 Iteration 3: Isolation Forest (`isolation_forest.py`) . . . 9
7.4 Iteration 4 (Final): Two-Stage IF + CUSUM (`cusum_if.py`) . . . 10

8 CUSUM: The Temporal Confirmation Stage **10**
8.1 What CUSUM Is . . . 10
8.2 Intuition Behind Each Parameter . . . 10
8.3 Physical Motivation for CUSUM in This Context . . . 11
8.4 CUSUM Parameter Tuning . . . 11

9 The Complete Two-Stage Pipeline **11**
9.1 Offline Training Phase . . . 11
9.2 Online Detection Phase (per timestamp t) . . . 11

10 Results and Quantitative Comparison **12**
10.1 Main Four-Way Comparison (868 Samples) . . . 12
10.2 CUSUM Parameter Results . . . 12

11 Ablation Study: Attack Intensity Sweep (`ablation.py`) **12**
11.1 Setup . . . 12
11.2 Results Table . . . 13
11.3 Key Ablation Findings . . . 13

12 Complete File-by-File Breakdown **13**
12.1 `src/data/download_acn.py` . . . 13
12.2 `src/data/preprocess.py` . . . 13
12.3 `src/data/dataset.py` . . . 13
12.4 `src/models/narx.py` . . . 14
12.5 `src/train/train.py` . . . 14
12.6 `src/eval/evaluate.py` . . . 14
12.7 `src/eval/isolation_forest.py` . . . 14
12.8 `src/eval/cusum_if.py` . . . 15
12.9 `src/eval/ablation.py` . . . 15

13 Frequently Asked Professor Questions **15**

14 Discussion Points and Limitations **16**
14.1 Why Recall = 1.0 Is the Design Goal . . . 16
14.2 Limitations . . . 16

15 Summary Cheat Sheet **17**

1 The Problem: What Is Being Solved?

1.1 Threat Context

Electric Vehicle (EV) charging stations communicate over open network protocols (mainly OCPP — Open Charge Point Protocol). A **man-in-the-middle adversary** can intercept this communication and inject false values into the session parameters: for example, reporting that the EV has *more available charging time* than it actually does.

The **Charging Management System (CMS)** trusts this data and accordingly delivers more (or less) energy to the battery, causing:

- Battery over-charge or under-delivery
- Billing fraud (customer billed for energy not received)
- Physical safety risks from sustained over-voltage

The Core Attack: FDI on `minutesAvailable`

The attacker sets: $\mathrm{MA}_{\mathrm{attack}} = \mathrm{MA}_{\mathrm{real}} \pm \vartheta$ where ϑ is the **attack intensity** in minutes. A larger ϑ = more obvious deviation in energy delivered. The stealthy, dangerous case is small ϑ (1–5 minutes).

1.2 Why Existing Approaches Fail

- **Manual threshold rules:** Set too tight ⇒ endless false alarms; set too loose ⇒ subtle attacks invisible.
- **Supervised classifiers (LSTM, CNN, GRU):** Need labelled attack samples from real networks — nearly impossible to obtain. The paper's own comparison showed LSTM achieved 0% precision and recall on this dataset.
- **Model-based residuals:** Assume linear, fully-observable systems — EV charging is non-linear and multi-modal.

1.3 The Solution Paradigm: Estimate Then Detect

Instead of classifying attacks directly, the project uses a **two-phase approach**:

1. **Estimation:** Train a NARX neural network on clean, normal charging data. It learns to predict how much energy *should* be delivered at each 5-minute interval.
2. **Detection:** Compute the **Error of Estimation**: $\mathrm{EoE}(t) = |y_{\mathrm{actual}}(t) - \hat{y}_{\mathrm{NARX}}(t)|$. Under a FDI attack, the actual delivery diverges from the NARX prediction, causing $\mathrm{EoE}(t)$ to spike. Flag these spikes.

This removes the need for any labelled attack data during training.

2 The Dataset: Caltech ACN-Data

2.1 Source and Content

- **Name:** Adaptive Charging Network (ACN) Data from Caltech campus, covering December 2020 – January 2021.
- **Type:** Real EV charging session records. Each row in the raw CSV is one charging session (from plug-in to unplug), not a time-series already.

- **Key fields per session:** connection time, done charging time, kWh delivered, station/site ID, user inputs (kWh requested, miles requested, minutes available, requested departure time, Wh per mile).

2.2 Preprocessing: From Sessions to Time-Series

The preprocessing step (`src/data/preprocess.py`) converts each single-row session into multiple 5-minute timestep rows:

1. **Parse timestamps:** Convert all datetime strings to UTC datetime objects.
2. **Extract user inputs:** The `userInputs` field is a JSON blob. Parse it to get `WhPerMile`, `milesRequested`, `minutesAvailable`, `kWhRequested`, `userID`, `requestedDeparture`.
3. **Remove incomplete sessions:** Drop rows missing `kWhRequested`, `minutesAvailable`, `connectionTime`, `doneChargingTime`, `kWhDelivered`.
4. **Derive target variable:** $\texttt{kWhDTS}(s) = \frac{\texttt{kWhDelivered}(s)}{\texttt{numTimeStamps}(s)}$ where $\texttt{numTimeStamps}(s) = \lceil \text{session_duration_min}$
5. **Explode into 5-minute rows:** Each session of n timestamps becomes n rows, each carrying the same session-level metadata but a `timestamps` counter $t \in [0, n-1]$.
6. **Split:** 70% → `acn_train_clean.csv`, 30% → `acn_estim_clean.csv` (sorted by `connectionTime` to preserve temporal order).

Numbers to remember

Paper target: 47,095 total samples (32,966 train + 14,129 estimation).
After preprocessing with the API's Dec 2020–Jan 2021 window, we get a smaller but proportional subset: 70/30 split is applied dynamically if under the target counts.

3 The NARX Neural Network — Deep Dive

3.1 What NARX Stands For

Nonlinear AutoRegressive network with eXogenous inputs.

- **Nonlinear:** The mapping function F is a neural network (nonlinear function), not a linear regression.
- **AutoRegressive (AR):** The network uses its own *past outputs* $y(t-1), y(t-2)$ as inputs. This creates a feedback loop — the model is *dynamic* (it has memory).
- **eXogenous inputs (X):** In addition to its own past outputs, it also receives *external* inputs $u(t-1), u(t-2)$ (the 15 charging session features).

The NARX equation is:

$$y(t) = F\big(u(t-1),\, u(t-2),\, y(t-1),\, y(t-2)\big) \tag{1}$$

where $u(t) \in \mathbb{R}^{15}$ is the exogenous input vector, $y(t) \in \mathbb{R}$ is the kWh delivered per 5-min stamp, and F is approximated by a small MLP.

3.2 Architecture Specification

Hyperparameter	Value
Input size	$15 \times m_x + m_y = 15 \times 2 + 2 = \mathbf{32}$
Hidden layers	1
Hidden neurons	10
Hidden activation	Sigmoid
Output neurons	1
Output activation	Linear (no activation)
Exogenous delay m_x	2 (uses $u(t-1)$ and $u(t)$)
Output delay m_y	2 (uses $\hat{y}(t-1)$ and $\hat{y}(t-2)$)
Weight init	Xavier uniform

Total parameters: $32 \times 10 + 10 + 10 \times 1 + 1 = 341$ parameters. This is intentionally tiny — it only needs to model smooth physical charging curves, not complex semantics.

3.3 The 15 Exogenous Features

Feature	Type	What it represents
`stationID`	Categorical (hashed)	Which EVSE unit
`siteID`	Categorical (hashed)	Which site/garage
`connectionTime_unix`	Timestamp	When EV was plugged in
`doneChargingTime_unix`	Timestamp	When charging finished
`kWhDelivered`	Numeric	Total kWh for session
`timestamps`	Integer offset	5-min step within session
`modifiedAt_unix`	Timestamp	Last session modification
`chargingCurrent`	Numeric	Pilot signal current (A)
`pilotSignal`	Numeric	EVSE pilot signal
`userID`	Categorical (hashed)	Which driver
`WhPerMile`	Numeric	Vehicle efficiency
`milesRequested`	Numeric	Distance requested
`minutesAvailable`	Numeric	**Attack target field**
`requestedDeparture_unix`	Timestamp	When driver leaves
`kWhRequested`	Numeric	Energy amount requested

`minutesAvailable` is the attack vector

This is the field that the FDI attacker modifies. The NARX is trained on legitimate values of this feature. When an attack inflates it by ϑ minutes, the model's prediction $\hat{y}$ diverges from actual delivery y, inflating $\mathrm{EoE}(t)$.

3.4 Open-Loop vs. Closed-Loop: The Critical Distinction

Open-Loop (Series-Parallel) — used during training: Feed *true* past labels $y(t-1), y(t-2)$ as the autoregressive inputs. This allows standard Mini-Batch Gradient Descent / Back-propagation because there is no recurrent dependency between batches. Computationally, the network is just an MLP with pre-assembled input vectors.

Closed-Loop (Fully-Recurrent) — used during inference: Feed the network's *own previous predictions* $\hat{y}(t-1), \hat{y}(t-2)$ back as inputs. The model now runs autonomously without needing ground-truth labels. Errors can accumulate but this mirrors real deployment where true labels are not available until after the session.

4 Core Concepts: Non-Linearity and Multi-Modality

Defining the Technical Terms

To understand why the NARX + Isolation Forest combo is superior, we must define the nature of the data we are handling.

4.1 Non-Linear Data in EV Charging

Definition: A relationship where the output does not change proportionally to the input (i.e., not a straight line).

- **Physics:** EV batteries use CCCV (Constant Current, Constant Voltage) charging. As the battery fills, the rate of energy delivery "tapers" off. This is a **non-linear decay**.
- **Implication:** Linear models (like Linear Regression) fail to capture the curve at the end of a session. The NARX network uses **Sigmoid activation functions** to mathematically model these physical "bends" in the charging profile.

4.2 Multi-Modal Data in FDI Detection

Definition: Data whose distribution has multiple "modes" or peaks. It means the data does not look like a single Bell Curve (Gaussian), but rather a series of different clusters.

- **Physics:** The Error of Estimation (EoE) behaves differently in different contexts:
  1. **Steady State:** Small errors while the car is charging normally.
  2. **Transitions:** Massive momentary spikes when a car plugs in/unplugs (cold-start artefacts).
  3. **Diversity:** Different EV models and session lengths create distinct "pockets" of error behavior.
- **Implication:** Simple statistical tools like **IQR** assume a single, unimodal distribution. If you use one threshold for all these modes, you get high False Positives. **Isolation Forest** is "Distribution-Agnostic"—it handles multi-modal data naturally because it isolates points based on their unique position, not their relation to a global average.

5 Technical Specifics: Weights and Gradients

5.1 Xavier (Glorot) Uniform Initialization

The "Just Right" Starting Point

In `src/models/narx.py`, weights are initialized using the Xavier Uniform method.

Concept: Neural networks are highly sensitive to their starting weights.

- **The Goal:** To keep the variance of the signal (and the gradients) stable as it passes through the layers. This prevents **Vanishing Gradients** (gradients getting too small to learn) or **Exploding Gradients** (gradients getting so large they break the model).
- **The Rule:** Weights are drawn from a uniform distribution $U[-a, a]$ where $a = \sqrt{6/(\text{fan_in} + \text{fan_out})}$.
- **Why it matters:** Since we use a **Sigmoid** activation, Xavier initialization is the industry standard to ensure the network starts training in a "healthy" state, allowing for faster and more stable convergence.

Why is this important to understand?

The NARX is trained open-loop (clean/fast) but deployed closed-loop (autonomous). This is the standard "series-parallel" training method for NARX networks and is exactly what the paper describes. A professor may ask: "Why not train closed-loop?" Answer: closed-loop training causes error accumulation to compound through backpropagation, making training numerically unstable.

5.2 NARX Input Window Construction (`dataset.py`)

For each time step t within a session, the input vector is:

$$\mathbf{x}(t) = \Big[\ \underbrace{u(t),\, u(t-1)}_{\text{exog window } (15\times 2)}\ ,\ \underbrace{y(t-1),\, y(t-2)}_{\text{output delays}}\Big] \in \mathbb{R}^{32}$$

Key rule: windows are built **per session**. The first $\max(m_x, m_y) = 2$ steps of each session are used as warmup seed values and *not* included as training targets. This prevents cross-session data leakage (using the last kWh values of session A as the "past outputs" of session B).

5.3 Why NARX for EV Charging?

EV charging energy draw at any 5-minute window depends on:

1. **Contextual instructions** given at session start (kWhRequested, minutesAvailable) — captured by exogenous inputs.
2. **The recent trajectory** of charging (battery fills up, current tapers) — captured by the autoregressive terms.

A pure MLP with no memory cannot model the tapering current of constant-current constant-voltage (CCCV) charging. A pure RNN (LSTM) requires labelled attack data to train as a classifier. NARX occupies the ideal middle ground: dynamic memory + clean regression training on normal data only.

6 Training Pipeline (`src/train/train.py`)

6.1 Training Hyperparameters

Setting	Value
Loss function	MSE (Mean Squared Error)
Optimizer	Adam, lr = 10^{-3}
Batch size	64
Max epochs	200
Early stopping patience	20 epochs
Val / Test split	15% / 15% (temporal, no shuffle)

6.2 Training Flow

1. Load `acn_train_clean.csv` and `acn_estim_clean.csv`.
2. Call `build_datasets()` which: (a) extracts the 15 features, (b) fits `MinMaxScaler` on training X and y, (c) transforms estimation set with the same scaler (no leakage), (d) builds NARX windows per session and returns `DataLoaders`.

3. Initialize `NARXNet(input_size=32, hidden_size=10)`.
4. Train loop: for each epoch, run forward pass, compute MSE loss, backward pass, update weights. Track validation loss.
5. Save best model weights to `checkpoints/narx_best.pt` when validation loss improves.
6. Early stop if validation loss does not improve for 20 consecutive epochs.
7. Save fitted scalers to `checkpoints/scalers.pkl`.

6.3 Target Benchmark

The paper reports an estimation-set MSE of $\approx 1.99 \times 10^{-5}$ (kWh/timestamp)^2. Our implementation achieves this benchmark, confirming a faithful replication of the NARX estimator.

7 Evolution of the Detection Layer

Once the NARX is trained, the **Error of Estimation** signal is:

$$\mathrm{EoE}(t) = \left| y_{\mathrm{actual}}(t) - \hat{y}_{\mathrm{NARX}}(t) \right|$$

Under a FDI attack, EoE(t) spikes. The question is: *how do we systematically flag those spikes?* The project went through four distinct approaches:

7.1 Iteration 1: Global IQR — The Paper Baseline (`evaluate.py`)

Algorithm:

1. Compute Q_1, Q_3, and IQR $= Q_3 - Q_1$ across *all* clean training EoE values globally.
2. Set **Lower Bound** LB $= Q_1 - 5 \cdot$ IQR and **Upper Bound** UB $= Q_3 + 5 \cdot$ IQR.
3. Flag a sample as a "spike" if EoE(t) $>$ UB (or $<$ LB, practically never happens for absolute error).
4. **Sliding window rule**: declare FDI attack at position t only if spikes occupy *all* positions in the window $[t-4, \ldots, t]$ (window length $q = 5$).

Rationale for $k = 5$, $q = 5$: Directly from the reference paper — set heuristically, not through any principled search.

Key Problems with Global IQR

1. **Session boundary contamination:** When NARX starts a new session, its delay buffer holds stale data from the previous session. This causes a large but harmless EoE spike for 1–2 timestamps. Global IQR absorbs these spikes into its statistics, either making UB too large (missing real attacks) or flagging all boundaries as attacks.
2. **Subtle attack blindspot:** For attacks with $\vartheta < 10$ minutes, the deviation in EoE(t) is smaller than $5 \times$ IQR. F1 $= 0$, Recall $= 0$ for $\vartheta \leq 10$.
3. **Heuristic parameters:** No principled method to choose $k = 5$ or $q = 5$.

Global IQR Results on 868-sample Estimation Set

TP=44, FP=9, FN=32, TN=783 $\Rightarrow$ Precision=0.830, Recall=0.579, F1=0.682

7.2 Iteration 2: Session-Aware IQR (`evaluate.py`)

Hypothesis: If session boundaries cause problems, compute IQR *independently for each session*. Each session gets its own LB_s and UB_s fitted on its own EoE values. Boundary spikes from other sessions cannot contaminate the thresholds.

Implementation: Group estimation EoE values by `sessionID`. For each group, compute percentiles and apply the same sliding window. Assign the session's LB/UB to all its samples.

Why it failed: The Caltech dataset's sessions, after 5-minute explosion, have a median length of only 15–20 timesteps. To compute stable quartiles you need at minimum 30–50 points. With 15 points, Q_1 and Q_3 are extremely sensitive to individual outliers — one session boundary spike makes the IQR enormous, setting an absurdly wide UB that misses all attacks, or vice versa.

Session-Aware IQR — Honest Negative Result

Performs *worse* than Global IQR across all attack intensities.
At $\vartheta = 60$ min: F1 = 0.025, Recall = 0.013 (versus 0.788/0.650 for Global IQR). This is reported as an important design lesson.

7.3 Iteration 3: Isolation Forest (`isolation_forest.py`)

Explain it Simply (The "Bad Apple" Analogy)

If you have 1,000 normal red apples and 5 weird blue apples, it takes many random questions to separate all the red apples from each other because they are similar. But it only takes one random chop ("Is it blue?") to isolate the weird ones. **Anomalies are few and different**, so they get isolated very quickly off from the rest of the data.
In this project, normal charging errors are packed tightly together. FDI attack errors are massive and different. The Isolation Forest randomly chops the data and instantly flags the errors that get isolated too quickly.

What Isolation Forest is mathematically:
An ensemble of binary trees, each built by:

1. Randomly selecting a feature dimension (here, just the scalar EoE value).
2. Randomly selecting a split value between the feature's min and max.
3. Recursively partitioning the data until each point is isolated or a depth limit is reached.

The **anomaly score** of a point is inversely proportional to its average path length across the forest:

$$s(x, n) = 2^{-\overline{h}(x)/c(n)}$$

where $\overline{h}(x)$ is the mean path length and $c(n)$ is the expected path length for a random binary tree of size n. Anomalies are isolated faster (shorter paths) $\Rightarrow$ lower average path length $\Rightarrow$ score closer to 1.

In practice, scikit-learn's `decision_function` returns negative values for anomalies: $d(x) < 0 \Rightarrow$ anomaly.

How it is used here:

1. Compute clean training EoE (no attacks) from the NARX predictions on the training set.
2. Grid-search the `contamination` hyperparameter (fraction of outliers expected) on a held-out validation slice with injected attacks, maximising F1.
3. Fit the final IsolationForest on all clean training EoE with the best contamination ($\xi = 0.01$).

4. Score the attack-injected estimation EoE: flag all points with $d < 0$.

Advantages over IQR:

- Makes **no distributional assumption** (IQR implicitly assumes symmetric, unimodal data).
- Handles **multi-modal EoE** (short sessions vs. long sessions produce very different error magnitudes).
- Session boundary spikes do not "contaminate" a learned model the way they inflate a global percentile.

Isolation Forest Alone Results

TP=76, FP=211, FN=0, TN=581 ⇒ Precision=0.265, Recall=**1.000**, F1=0.419
Perfect recall — catches every single attack — but 211 false positives.

The residual problem: IF scores each point independently. It cannot tell the difference between a real sustained attack and a one-off harmless session-boundary cold-start spike. Both look like outliers to a per-point scoring function. This is why IF alone has such high FP.

7.4 Iteration 4 (Final): Two-Stage IF + CUSUM (`cusum_if.py`)

Core Insight: FDI attacks and benign session-boundary spikes differ not in their *magnitude* but in their *temporal persistence*:

Event	EoE Signature	Duration
Session boundary cold start	Single large spike	1–2 timestamps
FDI attack ($\vartheta = 5$ min)	Sustained elevation	≥ 1 timestamp
FDI attack ($\vartheta = 30$ min)	Sustained elevation	≥ 6 timestamps

The approach: Use IF as a high-recall "wide net" (Stage 1), then use CUSUM as a "temporal filter" that only confirms sustained, persistent deviations (Stage 2). The final label is:

$$\text{ATTACK}(t) = \delta_{\text{IF}}(t) \ \wedge \ \delta_{\text{CUSUM}}(t)$$

8 CUSUM: The Temporal Confirmation Stage

8.1 What CUSUM Is

Cumulative Sum (CUSUM) Control Chart — a classical tool from statistical process control (Page, 1954). It accumulates the excess of the monitored signal above a reference level k (the "drift allowance"):

$$S(t) = \max\big(0,\ S(t-1) + |\text{EoE}(t)| - k\big), \quad S(0) = 0 \tag{2}$$

An alarm is triggered when $S(t) > h$ (the "alarm threshold"). After each alarm, S is **reset to zero** to prevent permanent post-detection locking.

8.2 Intuition Behind Each Parameter

k **(drift allowance):** Set it near the expected normal EoE level (μ_{clean}). Under normal operation, $|\text{EoE}(t)|$ fluctuates around k, so $S(t) = \max(0, S(t-1) + \text{small noise} - k)$ stays near zero (the $-k$ drains away any accumulation). Under an attack, $|\text{EoE}(t)|$ is consistently above k, so $S(t)$ grows monotonically each timestep.

h **(alarm threshold):** Controls the tradeoff between detection speed and false alarm rate. Low h: alarms fast but triggers on more noise. High h: only alarms on sustained deviations.

8.3 Physical Motivation for CUSUM in This Context

Why CUSUM Specifically?

Under a FDI attack that curtails charging by ϑ minutes, the actual energy delivery $y(t)$ drops while the NARX model (trained on normal behaviour) continues to predict non-zero values. This creates a **prolonged, directional downward deviation in** $\mathrm{EoE}(t)$ lasting for multiple consecutive 5-minute timestamps.
CUSUM was designed precisely for this signature: a small but *persistent* offset from baseline. A single-sample threshold (IQR, IF) must set the bar high enough to miss this subtle signal. CUSUM does not need any single sample to be extreme — it accumulates small deviations until their sum is undeniable.

8.4 CUSUM Parameter Tuning

Parameters (k, h) are chosen by grid search on a validation slice:

- k candidates: $\{0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 2.0\} \times \mu_{\text{clean}}$
- h candidates: $\{1, 2, 3, 4, 5, 6, 8, 10, 12, 15\} \times \sigma_{\text{clean}}$
- Objective: maximise F1 on val slice *subject to* Recall ≥ 0.90.

Selected values: $k^* = 0.00617$, $h^* = 0.03508$ (val F1 $= 0.886$). These satisfy $k^* \approx 2\mu_{\text{clean}}$ and $h^* \approx 15\sigma_{\text{clean}}$.

9 The Complete Two-Stage Pipeline

9.1 Offline Training Phase

1. Train NARX on clean labelled sessions (open-loop).
2. Run NARX on clean training set $\rightarrow$ collect $\{\mathrm{EoE}^{\text{clean}}(t)\}$.
3. Fit IsolationForest on $\{\mathrm{EoE}^{\text{clean}}(t)\}$ with $\xi = 0.01$.
4. Grid-search (k, h) by CUSUM on held-out 15% validation slice with injected attacks; select by max F1 subject to Recall ≥ 0.90.

9.2 Online Detection Phase (per timestamp t)

1. NARX produces $\hat{y}(t)$ via closed-loop inference.
2. Compute $\mathrm{EoE}(t) = |y(t) - \hat{y}(t)|$.
3. **Stage 1:** Query IF: $\delta_{\text{IF}}(t) = \mathbf{1}[d_{\text{IF}}(\mathrm{EoE}(t)) < 0]$.
4. **Stage 2:** Update CUSUM: $S(t) = \max(0, S(t-1) + |\mathrm{EoE}(t)| - k)$. Set $\delta_{\text{CUSUM}}(t) = \mathbf{1}[S(t) > h]$. If alarmed, reset $S(t) \leftarrow 0$.
5. **Final:** $\mathrm{ATTACK}(t) = \delta_{\text{IF}}(t) \wedge \delta_{\text{CUSUM}}(t)$.

Why AND Logic Eliminates Both Types of False Positives

- **IF flags but CUSUM doesn't:** The EoE spike only lasted 1–2 steps (session boundary). CUSUM's $S(t)$ bumped up but did not cross h. → Correctly classified as Normal.
- **CUSUM crosses h but IF doesn't flag:** Very slow, low-magnitude drift in a clean session (not an outlier to IF). → Correctly classified as Normal.
- **Both flag:** Sustained, large-magnitude EoE elevation. → True FDI attack confirmed.

10 Results and Quantitative Comparison

10.1 Main Four-Way Comparison (868 Samples)

The estimation set (868 samples: 792 normal, 76 attack-injected) produces:

Method	TP	FP	FN	TN	Acc	Prec	Rec	F1
Global IQR (baseline)	44	9	32	783	0.953	0.830	0.579	0.682
Isolation Forest alone	76	211	0	581	0.757	0.265	1.000	0.419
CUSUM alone	76	80	0	712	0.908	0.487	1.000	0.655
IF + CUSUM (proposed)	**76**	**47**	**0**	**745**	**0.946**	**0.618**	**1.000**	**0.764**

Key talking points for your professor:

- FN = 0 for proposed method: no attack is ever missed.
- FP reduced from 211 (IF alone) to 47 (IF + CUSUM): AND logic cuts false positives by 78%.
- The proposed method outperforms Global IQR on F1 (+12%) despite lower precision, because recall = 100% eliminates 32 missed attacks.
- In a safety context, FN cost >> FP cost: a missed attack is catastrophic; a false alarm is an operator inconvenience.

10.2 CUSUM Parameter Results

- Normal EoE stats: $\mu_{\text{clean}} = 0.00308$, $\sigma_{\text{clean}} = 0.00234$ (kWh/timestamp).
- Selected: $k^* = 0.00617 \approx 2\mu_{\text{clean}}$, $h^* = 0.03508 \approx 15\sigma_{\text{clean}}$.
- Validation F1 during tuning: 0.886.

11 Ablation Study: Attack Intensity Sweep (`ablation.py`)

11.1 Setup

The ablation sweeps $\vartheta \in \{1, 5, 10, 20, 30, 60\}$ minutes. For each ϑ, the attack scale factor is:

$$\text{scale}(\vartheta) = 1 + \vartheta/60$$

(60 = median session length in minutes). The attacked delivery is $y_{\text{attacked}}(t) = y_{\text{true}}(t) \times \text{scale}(\vartheta)$ for samples within the attack burst. The same 3 detectors are compared: Global IQR, Session-Aware IQR, Isolation Forest.

11.2 Results Table

ϑ (min)	Global IQR F1	Global IQR Rec	Session IQR F1	Session IQR Rec	Isolation Forest F1	Isolation Forest Rec
1	0.000	0.000	0.000	0.000	0.182	0.613
5	0.000	0.000	0.000	0.000	0.275	0.975
10	0.000	0.000	0.025	0.013	0.281	1.000
20	0.118	0.062	0.025	0.013	0.281	1.000
30	0.571	0.400	0.025	0.013	0.281	1.000
60	0.788	0.650	0.025	0.013	0.281	1.000

11.3 Key Ablation Findings

1. **Global IQR is completely blind for $\vartheta < 10$.** Attacks that manipulate `minutesAvailable` by fewer than 10 minutes produce EoE deviations below the 5×IQR bound at every point — F1 = 0, Recall = 0.

2. **Isolation Forest detects from $\vartheta = 1$.** With Recall = 0.613 at $\vartheta = 1$ and Recall = 1.0 for $\vartheta \geq 10$, IF is the only method that handles subtle attacks, validating its use as Stage 1.

3. **Session-Aware IQR is an honest negative result.** Worse than Global IQR at all intensities. Root cause: too few samples per session (15–20) to compute stable quartiles.

12 Complete File-by-File Breakdown

12.1 `src/data/download_acn.py`

Fetches raw JSON session records from the Caltech ACN-Data API (`ev.caltech.edu/dataset`). Saves them as `data/raw/sessions.csv`. Not run after initial data download.

12.2 `src/data/preprocess.py`

Input: `data/raw/sessions.csv`
Output: `data/processed/acn_train_clean.csv`, `acn_estim_clean.csv`
Steps:

1. Parse datetime columns to UTC.
2. Extract the nested JSON `userInputs` field.
3. Drop sessions missing `kWhRequested`/`minutesAvailable`.
4. Derive `kWhDeliveredPerTimeStamp` as session total divided by number of 5-min steps.
5. Explode each session into per-timestep rows.
6. 70/30 temporal split.

12.3 `src/data/dataset.py`

Purpose: Build NARX training windows and PyTorch DataLoaders.
Key functions:

- `prepare_features()`: Converts categorical fields (stationID, siteID, userID) to hash-based integers; extracts the 15 exogenous columns.

- `build_narx_windows_per_session()`: Builds input vectors $\mathbf{x}(t) = [u(t), u(t-1), y(t-1), y(t-2)]$ within each session only — prevents cross-session leakage.
- `build_datasets()`: Fits MinMaxScaler on train, transforms estim, returns loaders and scalers.

12.4 `src/models/narx.py`

Purpose: PyTorch implementation of NARXNet.
Key methods:

- `forward(x)`: Open-loop forward pass — MLP with pre-assembled 32-dim input vector. Returns (batch, 1).
- `closed_loop_predict(X_seq, y_seed, mx, my)`: Autonomous multi-step prediction. Maintains a circular buffer of past predictions; feeds them back as autoregressive inputs for each new timestamp.

12.5 `src/train/train.py`

Main training script. Loads processed CSVs, calls `build_datasets`, initializes NARXNet, runs training loop with Adam optimizer and MSE loss, applies early stopping, saves best weights and scalers.

12.6 `src/eval/evaluate.py`

Implements:

- `compute_iqr_bounds(eoe, k)`: Returns (LB, UB).
- `flag_spikes(eoe, lb, ub)`: Boolean array where EoE is outside bounds.
- `sliding_window_declare(spikes, q)`: Declares FDI at position t if all q prior positions are spikes.
- `inject_fdi_attacks(y_true, fraction, burst_len, scale_range, seed)`: Synthetic attack injection — picks random burst positions, scales their EoE by 3–8×.
- `session_aware_iqr()`: Per-session IQR variant.
- `evaluate()` and `evaluate_session_aware()`: Full pipeline runners with metrics and confusion matrices.

12.7 `src/eval/isolation_forest.py`

Implements:

- `tune_contamination()`: Grid-searches `contamination` on a validation slice.
- `evaluate_isolation_forest()`: Full IF pipeline: train on clean EoE, score attacked EoE, compute metrics.
- Three-way comparison: Global IQR vs. Session IQR vs. IF.

12.8 `src/eval/cusum_if.py`

The final proposed method. Implements:

- `cusum(eoe, k, h)`: Basic upper one-sided CUSUM.
- `cusum_reset(eoe, k, h)`: CUSUM with alarm-triggered reset (prevents permanent locking).
- `tune_cusum()`: Grid-searches (k, h) on val slice, maximising F1 subject to Recall ≥ 0.90.
- `evaluate_if_cusum()`: Full two-stage pipeline: fits IF on clean EoE, tunes CUSUM, evaluates AND combination.
- `plot_cusum_if()`: 5-panel figure for paper figures.

12.9 `src/eval/ablation.py`

- `inject_fdi_theta(y_true, theta)`: Attacks scaled by ϑ (physical model of minutesAvailable spoofing).
- `run_ablation()`: For each ϑ, scores with all three detectors and records F1/Recall/Precision.
- `plot_ablation()`: Two-panel figure (F1 left, Recall right) across ϑ sweep.

13 Frequently Asked Professor Questions

Q1: Why use a NARX and not an LSTM? LSTM is typically used as a *classifier* trained on labelled attack data — which we don't have. NARX is trained as a *regression model* on normal data only, which is abundant. The paper's experiments confirmed LSTM achieved 0% precision/recall when applied as a direct attack classifier on this dataset.

Q2: Why not train the NARX in closed-loop? Closed-loop training backpropagates through the feedback connections, causing error accumulation through the recursion. This makes the loss landscape very rough and training numerically unstable. The "series-parallel" (open-loop) training method is the standard approach for NARX (also used by the original paper).

Q3: Why is injection of attacks synthetic? Real FDI attack data from live charging networks is not available. The synthetic injection (multiply delivered kWh by a scale factor proportional to ϑ) faithfully models the physical effect of an attacker inflating `minutesAvailable`: the CMS thinks the car needs more time, so it continues delivering energy beyond what the battery actually needs, creating an energy surplus compared to the NARX expectation.

Q4: Why not use a global threshold after IF? IF already captures all attacks (100% recall). The problem is it also captures every session-boundary spike (high FP). We need a method that specifically penalises transient, momentary spikes while rewarding sustained deviations. Only a *temporal* method like CUSUM can do this — a static threshold on the IF scores would not distinguish between a 1-step spike and a 10-step attack.

Q5: Why is F1 only 0.764 if recall is perfect? Because Precision = 0.618 (47 false positives out of 123 predicted positives). F1 is the harmonic mean of precision and recall. With FN = 0, F1 = $2P/(P+1)$. Eliminating FP further would require either more CUSUM filtering (at the risk of missing subtle attacks) or better IF calibration. The current operating point was chosen to guarantee Recall = 1.0.

Q6: What is the physical timescale of an attack? At 5-minute resolution, $\vartheta = 5$ minutes corresponds to exactly 1 extra timestamp of false charging. $\vartheta = 30$ min $\Rightarrow$ 6 fraudulent timestamps. CUSUM, even with $\vartheta = 5$ min, accumulates the deviation over that 1 timestamp and crosses h because the deviation magnitude is large enough. Global IQR cannot see this because 1 timestamp fails the sliding window requirement of $q = 5$ consecutive spikes.

Q7: Does the method work on real-time data? Yes. The online detection phase is a per-timestamp computation with $O(T)$ complexity where T is the number of trees:

- NARX forward pass: $O(32 \times 10 + 10) \approx 330$ operations.
- IF scoring: $O(T \times \text{depth}) \approx O(200 \times 8)$.
- CUSUM update: $O(1)$.

Total latency per 5-minute interval is negligible (< 1 ms on CPU).

14 Discussion Points and Limitations

14.1 Why Recall = 1.0 Is the Design Goal

In safety-critical EV infrastructure, the cost asymmetry is:

- **Missed attack (FN):** Adversary disrupts sessions undetected. Direct harm to vehicles and financial loss.
- **False alarm (FP):** Human operator reviews a session that turns out to be benign — an inconvenience, not a failure.

With FPR = 5.9% (47 out of 792 normal sessions flagged), the system generates approximately 1 false alarm per 17 normal sessions. This is operationally manageable in a supervised alert dashboard.

14.2 Limitations

1. **Warm-up requirement:** CUSUM tuning needs a clean EoE baseline from normal operation. Unavailable for a newly deployed network.
2. **Concept drift:** If EV models change (e.g., new CCCV profiles), NARX residuals shift and (k, h) must be recalibrated.
3. **Attack type coverage:** Only `minutesAvailable` manipulation is tested. Attacks on `pilotSignal` or `kWhRequested` may have different EoE signatures.
4. **Dataset scale:** Our evaluation subset (868 samples after windowing) is smaller than the paper's full 14,129-sample estimation split due to API rate limiting.

15 Summary Cheat Sheet

What to Remember for the Presentation

Problem: FDI attacks manipulate EV charging parameters; subtle attacks ($\vartheta < 10$ min) are invisible to static thresholds.
Solution: Train NARX on normal data only $\rightarrow$ compute $\text{EoE}(t) = |y - \hat{y}|$ $\rightarrow$ run IF+CUSUM on EoE.
NARX: 1 hidden layer, 10 neurons, sigmoid, input size 32 (15 features $\times$ 2 delays + 2 output delays). Open-loop train, closed-loop infer.
Stage 1 (IF): No distributional assumption. Scores each $\text{EoE}(t)$ independently. Catches everything (Recall = 1.0) but many FP.
Stage 2 (CUSUM): $S(t) = \max(0, S(t-1) + |\text{EoE}| - k)$. Accumulates sustained drift; ignores 1-step spikes. AND with IF $\rightarrow$ kills >78% of FP.
Final result: F1=0.764, Recall=1.000, FN=0 (zero missed attacks), FP=47 (5.9% FPR). Outperforms Global IQR (F1=0.682, Recall=0.579).